SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I ***B Reddy Balu (192312499)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



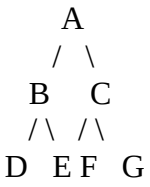
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



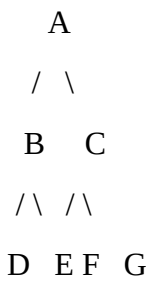
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1	**2**	**3**
4	**5**	**6**
7	**8**	**0**

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Question 1 – Breadth-First Search (BFS)

Graph edges: A–B, A–C, B–D, B–E, C–F, C–G. BFS uses a FIFO queue; children are added left to right. In each iteration the front node is removed, marked visited, and its unvisited children are enqueued.

Iteration	Queue (after the iteration)	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	(empty)	G

Answer 1 – BFS traversal order: A → B → C → D → E → F → G

Answer 2 – Queue contents: Start [A]; after iterations 1–7 the queue is [B, C], [C, D, E], [D, E, F, G], [E, F, G], [F, G], [G], [ ] (see table).

Question 2 – Depth-First Search (DFS)

DFS uses a LIFO stack. Children are pushed right to left so that the left child is on top and is explored first. The stack is written bottom → top (the rightmost item is the top).

Iteration	Stack (after the iteration, bottom → top)	Visited Node
1	C, B	A
2	C, E, D	B
3	C, E	D
4	C	E
5	G, F	C
6	G	F
7	(empty)	G

DFS traversal order: A → B → D → E → C → F → G

Note: DFS goes as deep as possible along one branch (A → B → D) before backtracking to E and then to the right subtree, whereas BFS visits the tree level by level.

Question 3 – 8-Puzzle using BFS

Initial state (S0) **Goal state**

1 3 6 5 _ 2 4 7 8	1 2 3 4 5 6 7 8 _

Notation: _ is the blank. Moves are the movement of the blank, tried in the order Up, Down, Left, Right. The state just removed from the queue is the one expanded; the parent state is not regenerated. States are named by the node that produced them.

Solvability check: the number of inversions in the initial state is 6 (even), and the goal has 0, so the goal is reachable.

State key (first 5 iterations)

S0: 1 3 6 5 _ 2 4 7 8	A1: 1 _ 6 5 3 2 4 7 8	A2: 1 3 6 5 7 2 4 _ 8	A3: 1 3 6 _ 5 2 4 7 8
A4: 1 3 6 5 2 _ 4 7 8	B1: _ 1 6 5 3 2 4 7 8	B2: 1 6 _ 5 3 2 4 7 8	B3: 1 3 6 5 7 2 _ 4 8
B4: 1 3 6 5 7 2 4 8 _	B5: _ 3 6 1 5 2 4 7 8	B6: 1 3 6 4 5 2 _ 7 8	B7: 1 3 _ 5 2 6 4 7 8
B8: 1 3 6 5 2 8 4 7 _			

A1–A4 are the children of S0 (Up, Down, Left, Right). B1–B8 are the children of A1–A4 respectively (two new children each).

BFS trace

Iter	Current State (expanded)	Queue Before Expansion	Queue After Expansion
1	S0	S0	A1, A2, A3, A4
2	A1 (blank Up)	A1, A2, A3, A4	A2, A3, A4, B1, B2
3	A2 (blank Down)	A2, A3, A4, B1, B2	A3, A4, B1, B2, B3, B4
4	A3 (blank Left)	A3, A4, B1, B2, B3, B4	A4, B1, B2, B3, B4, B5, B6
5	A4 (blank Right)	A4, B1, B2, B3, B4, B5, B6	B1, B2, B3, B4, B5, B6, B7, B8

Details: S0 has the blank in the centre, so it has 4 moves. A1 (blank top-middle) generates B1 (Left) and B2 (Right); A2 (blank bottom-middle) generates B3 (Left) and B4 (Right); A3 (blank middle-left) generates B5 (Up) and B6 (Down); A4 (blank middle-right) generates B7 (Up) and B8 (Down). The goal is not among these states, so BFS continues level by level until depth 8.

Answers

1. Node expanded in each iteration: Iteration 1 – S0; 2 – A1; 3 – A2; 4 – A3; 5 – A4. BFS expands the states in the order they were generated (FIFO): the depth-0 state, then all depth-1 states, then depth-2 states, and so on.

2. Total states generated: In the 5 iterations traced above, 12 new states are generated (4 + 2 + 2 + 2 + 2), i.e. 13 states including S0. Running BFS to completion (operator order Up, Down, Left, Right, with repeated states discarded) the goal is found at depth 8; the goal is removed from the queue on the 311th expansion, by which time 499 new states have been generated (500 distinct states including S0). The exact count depends on the operator order and on whether the goal test is applied at generation or at expansion.

3. Solution path (8 moves, blank moves: Right, Up, Left, Down, Left, Down, Right, Right):

Step 0: Start 1 3 6 5 _ 2 4 7 8	Step 1: Right (swap with 2) 1 3 6 5 2 _ 4 7 8	Step 2: Up (swap with 6) 1 3 _ 5 2 6 4 7 8

Step 3: Left (swap with 3) 1 _ 3 5 2 6 4 7 8	Step 4: Down (swap with 2) 1 2 3 5 _ 6 4 7 8	Step 5: Left (swap with 5) 1 2 3 _ 5 6 4 7 8
Step 6: Down (swap with 4) 1 2 3 4 5 6 _ 7 8	Step 7: Right (swap with 7) 1 2 3 4 5 6 7 _ 8	Step 8: Right (swap with 8) – GOAL 1 2 3 4 5 6 7 8 _

Path as tiles moved: 2 → 6 → 3 → 2 → 5 → 4 → 7 → 8 (the blank moves Right, Up, Left, Down, Left, Down, Right, Right). Path cost = 8.

4. Why BFS finds the shortest solution: Every move in the 8-puzzle has the same cost (1), so the graph is unweighted. BFS expands all states at depth d before any state at depth d+1, because the FIFO queue keeps nodes ordered by depth. Therefore the first time the goal is reached, it is at the smallest possible depth; any shorter solution would already have been found at an earlier level. BFS is also complete (the branching factor is finite) and the solution it returns is optimal (here, 8 moves). The trade-off is memory: it must store every state of the current frontier, which grows exponentially.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
Criteria	**Weightage (%)**	**Level 4 – Exemplary (4)**	**Level 3 – Proficient (3)**	**Level 2 – Developing (2)**	**Level 1 – Beginning (1)**
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.